SEAGLASS FOUNDRY

Forged For Speed. Refined by Craft.

ForgeGIS™

GPU-Accelerated Geospatial Compute for the Agent Era

ForgeGIS™ is a GPU-accelerated geospatial library for Java with a native Model Context Protocol server: 239 operations across terrain, hydrology, visibility, spectral and ML, geometry, point clouds and more, callable from a Java pipeline or by an AI agent — the same engine either way. Raster data stays on the GPU between pipeline stages, every operation falls back to the CPU, and it ships as Maven artifacts with no GDAL and nothing native to install. Release 1.2.6 is about being right: every measurement on the WGS84 ellipsoid, and outputs checked against GDAL.

239

operations across 11 categories

in the certified catalog; 237 reachable over MCP, and 170 compose into one GPU-resident pipeline call

8–79×

multi-stage pipelines vs GDAL CLI chains

4096² rasters, warm-mean — measured on 1.2.4, August 2026

WGS84

one geodesic engine for every measurement

new in 1.2.6: Karney's method on the ellipsoid for every length, area, distance and bearing

What it is

The compute engine underneath geospatial analysis: slope and viewshed, watersheds and flood paths, classification and interpolation, overlay and topology, routing and point clouds. Every operation is a Java method, and the same operations are exposed to AI agents through an MCP server — a thin layer over one implementation, so an agent's answer is the script's answer.

Why it matters

Today's geospatial stack was built for an analyst at a desktop, assembled from native libraries and command-line tools. Agents need primitives that are composable, callable and fast enough that a multi-step workflow does not time out — and regulated work needs those answers to be checkable. ForgeGIS provides both without a native build chain to maintain.

How it's built differently

Data stays GPU-resident across a pipeline instead of round-tripping through disk between stages — the structural reason multi-step analysis runs at machine speed. Branching graphs mix raster, vector and point-cloud stages, and existing QGIS models import directly. Deployment is Java: the GPU is reached through the JOCL OpenCL bindings, whose native bridge ships inside their jar — no GDAL, and no native binaries of ForgeGIS's own.

In the Forge suite. ForgeGIS is the suite's compute engine. The Seaglass Control Center™ installs it with a private Java runtime, ForgeMind™ runs it as an MCP tool server and asks it for analysis, ForgeData™ finds the data it reads, and ForgeGIS Studio™ and Seaglass Globe™ draw its results. Outside the suite it is a set of Maven artifacts any JVM service can depend on.

CONTINUED ON REVERSE →

WHO FORGEGIS IS FOR

Defense, intelligence and aerospace programs

ForgeGIS reaches an air-gapped or accredited host as Java archives — no GDAL, no system-level native install, no ForgeGIS native binaries to sign — and needs only a JDK and a GPU driver there. Viewshed, intervisibility, Fresnel clearance and line of sight run on the GPU, and every measurement is on the WGS84 ellipsoid.

Builders of AI agents for geospatial work

An agent starts with nine tools — enough to list the operations and run a pipeline or a branching graph of them in one call — or opts into 84, including 75 typed tools for common tasks. Failures come back as typed reasons an agent can branch on, and a job that will not fit in memory is refused before it starts.

Geospatial SaaS, climate and agtech platforms

GPU compute without leaving the JVM: drop ForgeGIS into an existing Java service as a Maven dependency, with no CUDA lock-in — any OpenCL GPU, or the CPU. The figures on this page were measured on a single laptop with an RTX 5070, not a server.

PERFORMANCE — MEASURED ON 1.2.4, AUGUST 2026

8–79×

multi-stage pipelines against the same chain run as GDAL command-line tools, at 4096² rasters

8.9×

geometric mean over 226 operation records with a GDAL equivalent, no category excluded

208 / 226

records won against GDAL; all 18 losses on rasters of 1024² or smaller

All warm-mean, measured on ForgeGIS 1.2.4 in August 2026 on one laptop, in the last canonical benchmark run. Small single operations are not where the case rests: at 1024² and below GDAL wins a number of them, and we do not market those.

New in 1.2.6 — a release about being right. One geodesic engine on WGS84 behind every measurement. Outputs checked against GDAL and stated with their scope: contours within 1 cm of `gdal_contour` on real SRTM data, and NoData filling that defaults to GDAL's inverse-distance method. Every raster operation is held to CPU/GPU agreement, a NoData sweep and a US-survey-foot check. Interpolation and ground filtering now run true Sibson, thin-plate spline, Delaunay TIN and progressive morphological algorithms.

Changes an integrator must act on. `compute_haversine_distance` is renamed `compute_geodesic_distance`, with no alias; viewshed output is UINT8 (255 visible, 0 not visible) as `gdal_viewshed` writes it; point parameters on the MCP surface default to EPSG:4326.

Get the full Technical Brief

The Technical Brief covers the architecture, the agent surface, the accuracy evidence and its scope, the full capability catalog, benchmark methodology and deployment. ForgeGIS is proprietary and licensed by written agreement; evaluation builds are available to qualified teams.

rich@seaglassfoundry.com · seaglassfoundry.com

ForgeGIS™, ForgeMind™, ForgeData™, ForgeGIS Studio™, Seaglass Globe™, Seaglass Control Center™, and Seaglass Foundry™ are trademarks of Seaglass Foundry LLC. GDAL, PDAL, JTS, GeographicLib, QGIS, OpenCL, NVIDIA, GeForce, RTX, Java, Apache Maven, and the Model Context Protocol are trademarks or marks of their respective owners; reference to them does not imply endorsement. © 2026 Seaglass Foundry LLC. All rights reserved.